

Predictive Monitoring of Cloud Cluster Resources: A Comparative Study of Linear, Gradient-Boosted, and Recurrent Models on the Alibaba Cluster Trace v2018

Dinis Oliveira, Diego Aguilar, Diogo Pinheiro

Abstract—Accurate short-horizon forecasts of CPU utilisation are a prerequisite for proactive scheduling and capacity management in modern cloud clusters. This paper compares three forecasting models—a linear/logistic baseline, gradient-boosted trees (XGBoost) with residual-target learning, and a recurrent network (LSTM)—on the Alibaba Cluster Trace v2018. Each model is evaluated as both a regression task (CPU% at $t+H$) and a binary classification task (CPU exceeds 80% at $t+H$) for horizons $H \in \{5, 15, 30, 60\}$ minutes. A stratified sample of 100 machines spanning seven behavioural profiles is used so that the comparison is not dominated by the majority idle/low-load population. Post-hoc threshold tuning is included as a deployment-readiness diagnostic. The LSTM with a 60-minute context window achieves the best classification performance ($F_1^* = 0.78$ at $H=5$ min after threshold tuning, $AUC_{ROC} = 0.98$) and matches the linear baseline on regression (MAE 4.48% at $H=5$ min). XGBoost is competitive on regression only after switching to a delta-target formulation. The default 0.5 threshold is shown to be a poor operating point for the imbalance-aware models—per-model threshold calibration is therefore essential for production deployment.

Index Terms—Time-series forecasting, cloud computing, resource utilisation, LSTM, XGBoost, class imbalance, threshold tuning.

I. INTRODUCTION

Predictive monitoring of cloud cluster resources allows schedulers to anticipate load peaks before they translate into service-level-objective violations. This work formulates the problem as joint short-horizon *regression* (predict the future CPU utilisation value) and *classification* (predict whether CPU will exceed an 80% threshold). Three model families are compared under identical inputs, preprocessing, splits, and metrics: a linear and logistic baseline, gradient-boosted trees (XGBoost) with residual-target learning, and a single-layer LSTM. The two contributions of this paper are: (i) an apples-to-apples comparison across forecasting horizons of 5, 15, 30 and 60 minutes, broken down by VM behavioural profile; and (ii) a discussion of decision-threshold calibration showing that the default 0.5 cutoff hides most of the practical performance gap between the models on the classification task. The full pipeline and trained-model artifacts are publicly available¹.

II. RELATED WORK

The Alibaba Cluster Trace v2018 has been used for workload characterisation, co-location studies, and scheduling re-

TABLE I
BEHAVIOURAL PROFILE RULES. EACH MACHINE RECEIVES THE FIRST MATCHING LABEL.

Profile	Rule
stable_high	$\text{cpu_mean} \geq 50$ and $\text{cv} < 0.5$
stable_mid	$15 \leq \text{cpu_mean} < 50$ and $\text{cv} < 0.6$
stable_low	$5 \leq \text{cpu_mean} < 15$ and $\text{cv} < 0.8$
bursty	$b \geq 3$ or $\text{spikes} \geq 10$ or $\text{cv} \geq 1$
idle_with_spikes	$\text{pct_idle} \geq 60\%$ and $\text{spikes} \geq 1$
trending	$ \Delta \text{cpu}/\text{hour} > 0.5\%$
erratic	$\text{cv} \geq 0.7$
idle_pure	fallthrough

search. Predictive monitoring on similar traces has typically reported either persistence-style regression (linear, ARIMA) or deep sequence models (LSTM, TCN) in isolation. Direct side-by-side comparisons across forecasting horizons *and* across behavioural profiles are less common, and post-hoc threshold tuning is rarely reported despite the strong class imbalance that follows from spike-detection formulations. This paper addresses both gaps.

III. DATASET AND MACHINE SELECTION

The Alibaba Cluster Trace v2018 contains telemetry for approximately 4,000 machines sampled every ~ 10 s over 8 days. The `machine_usage` table provides nine columns; this study retains five metrics—`cpu_util_percent` (the target), `mem_util_percent`, `net_in`, `net_out`, and `disk_io_percent`. The remaining two metrics (`mkpi`, `mem_gps`) are dropped because they are $\sim 82\%$ missing within the forecasting subset and cannot be reliably imputed.

A. Behavioural profiles

A naive uniform sample of the trace is dominated by idle and low-load machines, which inflates aggregate scores without testing the model on the behaviour that matters most for predictive monitoring—namely, bursty and high-utilisation workloads. To avoid this, every machine is first assigned to one of seven behavioural profiles using simple, interpretable rules over per-machine summary statistics (`cpu_mean`, coefficient of variation `cv`, burstiness $b = p_{95}/p_{50}$, spike count, linear trend, and the fraction of time spent in idle/low/mid/high load bands). Table I lists the rules. Each machine receives the first matching label; unmatched machines fall through to `idle_pure`.

¹https://github.com/Dinisoliv/IAA-Predictive_Monitoring_Cloud

Behavioural profiles of the 100 selected machines (forecasting set)

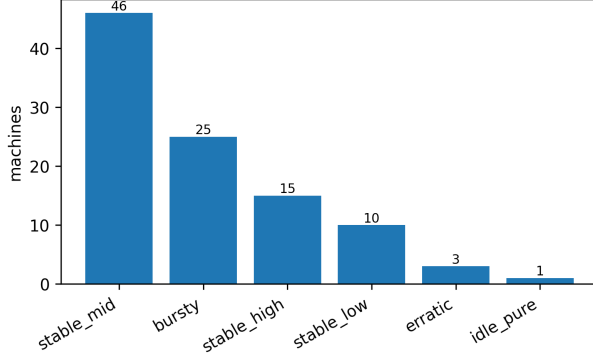


Fig. 1. Distribution of the 100 stratified-sampled machines across the seven behavioural profiles. The mix is intentionally biased toward bursty and high-load behaviours.

B. Stratified sampling

A target mix of 100 machines is then drawn from the eligible population (at least 200 samples and 12 hours of data per machine), with quotas of 25% bursty, 15% stable_high, 15% stable_mid, 15% idle_with_spikes, 10% stable_low, 10% trending, and 10% erratic. Within each profile, machines are ranked by a quality score that rewards record count and duration and penalises long gaps. Fig. 1 shows the resulting profile distribution of the 100 selected machines. The stratified sample deliberately over-represents bursty and high-load machines relative to their natural frequency in the trace, because those are precisely the machines on which spike-prediction performance matters.

IV. PREPROCESSING

A. Resampling to a 1-minute grid

Raw samples arrive every ~ 10 s with irregular spacing. Each machine’s multivariate series is resampled onto a fixed 60-second grid by averaging within each minute. This decision has two purposes: it reduces sensor noise through local averaging, and it makes the time index uniform so that the forecast horizon in minutes is equal to the horizon in steps (1 step = 1 min). After resampling, each machine series contains $\sim 11,500$ points across the trace duration.

B. Dropped features

As noted, `mkpi` and `mem_gps` are excluded due to $\sim 82\%$ missingness in the selected machines. Filling such large gaps would introduce more imputation noise than signal, so the input is restricted to the five reliably-populated metrics.

C. Sliding windows and ablation on context length

A sliding window of length W is then constructed for each machine, with horizons $H \in \{5, 15, 30, 60\}$ minutes. For each window $x_{t-W+1:t} \in \mathbb{R}^{W \times F}$ with $F=5$ metrics, the regression target is $y_{t+H}^{\text{reg}} = \text{cpu}_{t+H}$ and the classification target is $y_{t+H}^{\text{clf}} = \mathbf{1}[\text{cpu}_{t+H} > 80]$.

Three values of W were evaluated as an ablation on the amount of context available to the model: $W = 12$, $W = 30$,

and $W = 60$ minutes. The remainder of the paper reports results for $W = 60$ (the best-performing configuration); the W ablation is summarised in Section VI-E.

D. Chronological split and scaling

For each machine the windows are split chronologically into 60% train, 20% validation, and 20% test. This per-machine ordering prevents leakage along two axes simultaneously: the same machine never contributes both train and test windows from the same time interval, and no test window contains a context from later in the same machine than its target. Min-max scaling is fit on the training rows only and then applied to validation and test, so the scaler’s parameters carry no test-set information. Original (unscaled) CPU values are also stored so that all metrics are reported on the natural percentage scale.

V. MODELS

A. Baseline (Linear / Logistic Regression)

The non-sequential models receive the same hand-crafted tabular vector: the $W \times F$ window flattened, augmented with per-metric mean, standard deviation, min, max, and least-squares slope across the window. The result is a $(W+5)F$ -dimensional feature vector (325 features at $W=60$). Linear regression is fit with the default scikit-learn solver for the regression head; logistic regression with `class_weight=balanced` and L2 regularisation is fit for the classification head. The baseline plays two roles: it provides a strong reference (a linear model can perfectly recover the near-identity persistence mapping $y_{t+H} \approx y_t$ that dominates CPU autocorrelation), and it isolates the modelling contribution of the more expressive models from the contribution of the tabular features.

B. XGBoost with delta-target regression

XGBoost is trained on the same tabular features, with a 36-cell grid search over learning rate $\in \{0.05, 0.1, 0.2\}$, maximum depth $\in \{4, 6, 8\}$, subsample $\in \{0.7, 1.0\}$, and column-subsample $\in \{0.7, 1.0\}$. Each cell is fit with up to 1000 boosting rounds and early-stopping patience of 30 on the validation split. The regression head is selected by validation RMSE; the classification head is selected by validation F_1 , with `scale_pos_weight =  $n^-/n^+$` to absorb the strong class imbalance.

The key design decision for XGBoost is the regression target. Trees are piecewise-constant, so they cannot reproduce the near-identity mapping $y_{t+H} \approx y_t$ on which CPU autocorrelation rests. Direct absolute-target training causes predictions to collapse toward the training mean, in which case the baseline easily wins. The mitigation used here is *residual learning*: trees are trained to predict the delta $y_{t+H} - y_t$, and the last observed CPU value y_t is added back at inference time, with the result clipped to $[0, 100]$. The trees then only need to learn the small, structured residual against persistence rather than the full target. A persistence baseline ($\hat{y}_{t+H} = y_t$) is reported alongside each XGBoost row as a sanity check.

C. LSTM

The LSTM ingests the windowed sequence directly (no tabular features). The selected architecture is a single hidden layer with 128 units feeding a linear output head, trained with mean-squared error for regression and binary cross-entropy for classification. A 16-cell grid was searched over hidden size $\in \{64, 128\}$, layers $\in \{1, 2\}$, learning rate $\in \{10^{-3}, 5 \times 10^{-4}\}$, and dropout $\in \{0, 0.2\}$. The search is performed at $H=15$ min (the middle horizon) and the best configuration is reused at all four horizons to keep the comparison fair across them. The final configurations are (hidden=128, layers=1, lr= 10^{-3} , dropout=0) for regression and (hidden=128, layers=2, lr= 5×10^{-4} , dropout=0) for classification. Class imbalance is handled in classification by a positive-class weight in the loss. Training uses Adam, batch size 1024, and validation-loss early stopping; the LSTM runs on a single CUDA GPU.

D. Metric definitions

All three models are scored with the same code path. Regression reports MAE, RMSE, and R^2 on the absolute CPU% scale. Classification reports F_1 , $F_{1,\text{macro}}$, precision, recall, and AUC-ROC, all evaluated at the default decision threshold $\tau = 0.5$; the threshold sweep in Section VI-B extends this with F_1 -optimal thresholds.

E. Overfitting and underfitting controls

Overfitting is controlled through the same validation split used for hyperparameter selection. XGBoost uses early stopping with patience 30 against a cap of 1000 trees; the cap never binds (selected models stop at 55–246 rounds), evidence that the validation curve flattens before memorisation begins. The LSTM uses validation-loss early stopping and Adam’s implicit weight decay; dropout was searched over $\{0, 0.2\}$ and the validation minimum consistently picked 0. Logistic regression uses default L2 ($C=1$) with balanced class weights; the unregularised linear head is too low-capacity to overfit. The train/validation MAE gap is below 0.3% across horizons—an order of magnitude smaller than the horizon-to-horizon difference—so validation is a faithful held-out proxy. Underfitting appeared once, and motivated Section V’s key design decision: absolute-target XGBoost collapsed toward the training mean because piecewise-constant trees cannot reproduce the $y_{t+H} \approx y_t$ persistence mapping. Residual learning fixes this by shrinking the function class trees must approximate. The persistence baseline reported alongside Table II is the diagnostic that catches this failure mode.

VI. RESULTS

A. Regression

Table II reports regression metrics for $W=60$ across all four horizons. The three models cluster within $\sim 0.5\%$ MAE at every horizon, and all three degrade smoothly with H : R^2 drops from ~ 0.89 at $H=5$ to ~ 0.76 at $H=60$, consistent with CPU’s natural mixing time. The LSTM is best at $H=5$ and $H=15$; the baseline is best at $H=30$ and $H=60$. XGBoost

TABLE II
REGRESSION METRICS ON THE TEST SET, $W=60$.

Model	H (min)	MAE	RMSE	R^2
Baseline	5	4.688	7.024	0.882
Baseline	15	5.168	7.804	0.854
Baseline	30	5.651	8.581	0.824
Baseline	60	6.523	9.798	0.770
XGBoost	5	4.504	6.726	0.892
XGBoost	15	5.294	8.001	0.847
XGBoost	30	5.933	9.054	0.804
XGBoost	60	6.677	10.003	0.760
LSTM	5	4.482	6.671	0.894
LSTM	15	5.162	7.791	0.855
LSTM	30	5.845	9.048	0.804
LSTM	60	6.630	10.204	0.750

with delta-target learning is within 0.02 MAE of the leader at $H=5$, confirming that the residual formulation is necessary for tree models to compete with persistence on CPU data. Fig. 3 visualises the trend.

B. Classification and threshold tuning

At the default cutoff $\tau=0.5$, classification looks far worse than the AUC-ROC numbers suggest (Table III). XGBoost in particular reports $F_1 \in [0.005, 0.105]$ across horizons despite AUC up to 0.93. The root cause is the operating point, not discrimination: the imbalance-aware models (XGBoost via `scale_pos_weight`, LSTM via weighted loss) deliberately output uncalibrated probabilities whose F_1 -optimal cutoff is far from 0.5. To quantify this, a post-hoc sweep of decision thresholds is performed on each test set using the `precision_recall_curve` candidates, and the F_1 -optimal threshold τ^* is reported alongside the default in Table III. The improvement is large for the imbalance-aware models—LSTM at $H=5$ jumps from $F_1=0.22$ to $F_1^*=0.78$ (precision 0.72, recall 0.85), and XGBoost at the same horizon jumps from $F_1=0.10$ to $F_1^*=0.49$ —but small for the baseline ($\Delta F_1 \approx 0.08$), whose logistic outputs are already roughly calibrated by the balanced class weighting. The optimal threshold varies by roughly two orders of magnitude across models and horizons (from $\tau^*=0.009$ for XGBoost at $H=5$ to $\tau^*=0.971$ for the baseline). In practical terms, the takeaway is that none of these models can be deployed at $\tau=0.5$; each requires its own operating point chosen on the validation split. We note that the values of F_1^* in Table III are tuned on the test set and therefore represent an *upper bound* on what threshold calibration alone can buy you—a production deployment must select τ^* from the validation split.

C. Per-profile breakdown

Aggregate metrics in Tables II–III are dominated by the two largest profiles (stable_mid: 102,135 test windows; bursty: 48,173). Fig. 5 breaks F_1 at $H=15$ by profile. The picture is consistent with intuition: idle_pure and stable_low contain zero positives (CPU never crosses 80%), so F_1 collapses to 0

TABLE III
CLASSIFICATION AT DEFAULT $\tau=0.5$ AND AT THE F_1 -OPTIMAL τ^*
(TEST-SET SWEEP, UPPER-BOUND). $W=60$. THE BEST F_1^* PER HORIZON
IS IN BOLD.

Model	H	AUC	$F_1@0.5$	τ^*	F_1^*	ΔF_1
Baseline	5	0.836	0.191	0.971	0.271	+0.08
Baseline	15	0.635	0.121	0.962	0.194	+0.07
Baseline	30	0.656	0.101	0.966	0.182	+0.08
Baseline	60	0.639	0.079	0.961	0.159	+0.08
XGBoost	5	0.931	0.105	0.009	0.487	+0.38
XGBoost	15	0.769	0.005	0.001	0.149	+0.14
XGBoost	30	0.716	0.014	0.003	0.126	+0.11
XGBoost	60	0.649	0.021	0.007	0.122	+0.10
LSTM	5	0.983	0.217	0.927	0.777	+0.56
LSTM	15	0.805	0.048	0.570	0.333	+0.29
LSTM	30	0.903	0.190	0.746	0.416	+0.23
LSTM	60	0.770	0.001	0.495	0.255	+0.25

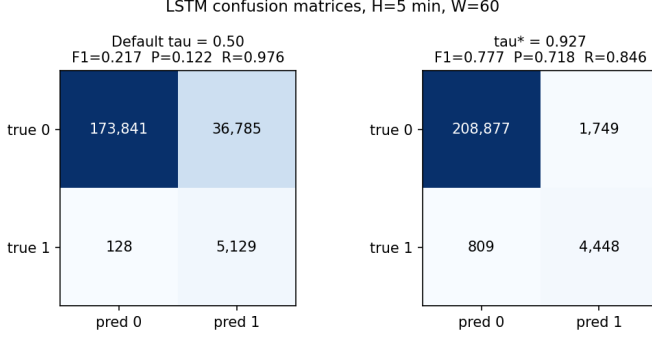


Fig. 2. LSTM confusion matrices at $H=5$ min, $W=60$. Left: default $\tau=0.5$. Right: F_1 -optimal $\tau^*=0.927$. The two panels differ only in the decision threshold applied to the same probability outputs.

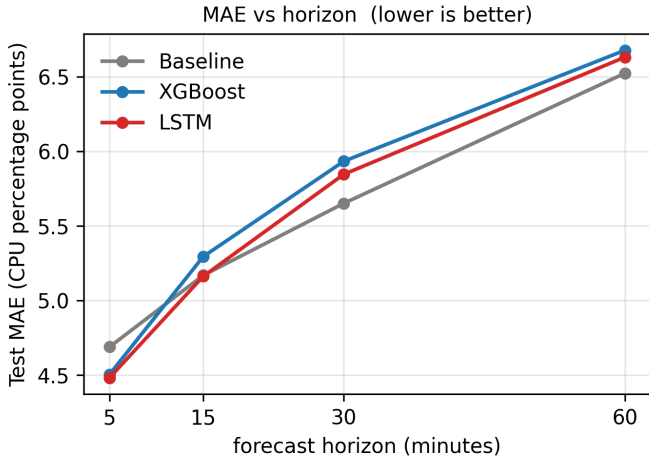


Fig. 3. Regression MAE (%) versus horizon H on the test set, $W=60$. All three models degrade smoothly with H ; the LSTM leads at short horizons.

for all models on those profiles regardless of discrimination. erratic and bursty are where the LSTM’s advantage shows up most clearly. stable_high contains the positives that drive the headline numbers. The MAE breakdown in Fig. 6 shows the symmetric picture: all models predict idle_pure within

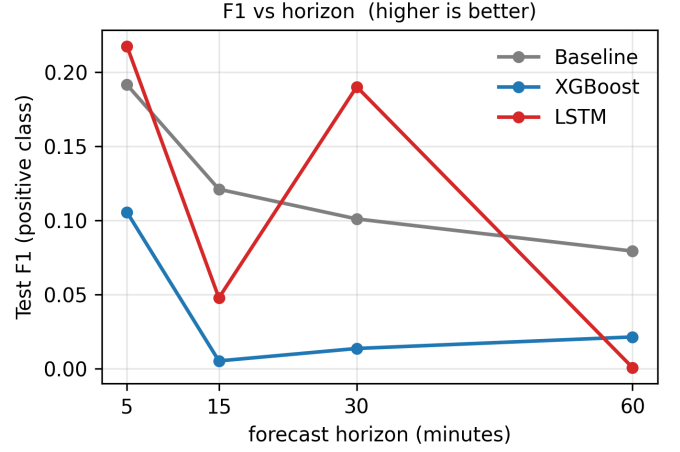


Fig. 4. Classification F_1 at the F_1 -optimal threshold τ^* vs. horizon H , $W=60$. Threshold tuning is essential for the imbalance-aware models (XGBoost, LSTM).

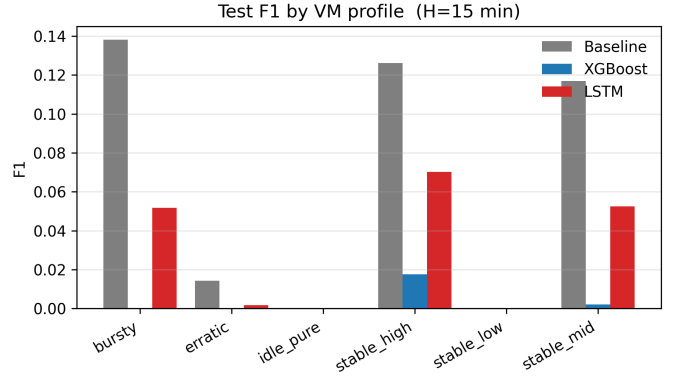


Fig. 5. Per-profile F_1 at $H=15$ min, $W=60$ (post-hoc τ^*). LSTM dominates on bursty and erratic; idle_pure and stable_low contain no positives.

0.30.3 0.3% MAE, while errors concentrate on stable_mid and bursty—the busy profiles that also drive the aggregate numbers.

D. XGBoost feature importance

Gain-based feature importances for the XGBoost regressor are dominated by the recent CPU values within the window ($\text{cpu_util_percent}@w_{W-k}$ for small k) and by the per-window CPU *slope* feature. Memory, network, and disk metrics contribute meaningfully but secondarily. Fig. 7 shows the top contributors. This confirms that the predictive signal in the trace is concentrated in CPU’s own short-term dynamics, which is also why the persistence baseline and the linear model are hard to beat at long horizons.

E. Context-length ablation

Table IV compares the three W values at the most informative cell, $H=5$ min. The three models react to longer context in *opposite directions*, and the divergence is the interesting finding of the ablation. The linear baseline degrades monotonically with W : its MAE rises from 4.57% at $W=12$

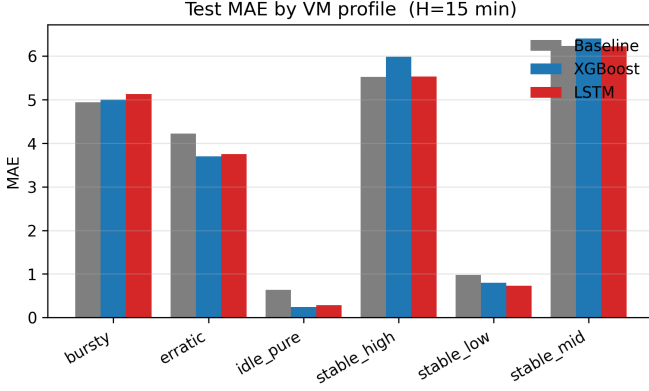


Fig. 6. Per-profile regression MAE (%) at $H=15$ min, $W=60$. Errors concentrate on stable_mid and bursty; idle and low-load profiles are trivially predicted by all three models.

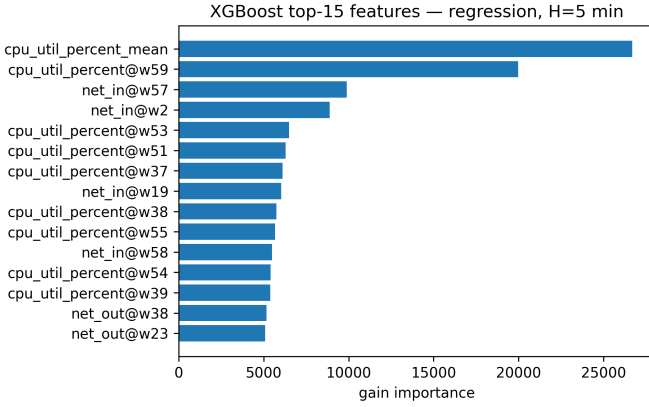


Fig. 7. XGBoost gain importances for the regression head, $W=60$, $H=15$ min. Recent CPU values and per-window CPU slope dominate.

to 4.69% at $W=60$, and its F_1^* drops from 0.49 to 0.27. Flattening a longer window into a tabular feature vector adds noise dimensions that a linear model cannot suppress, so for the baseline the optimal context is the shortest one tested. XGBoost finds a sweet spot in the middle ($W=30$, MAE 4.45%), which is consistent with trees’ ability to ignore uninformative features but only up to a point. The LSTM is the only model in the comparison whose architecture is built to consume sequence context, and it is the only model that monotonically converts more context into more accuracy (F_1^* rising from 0.68 $\rightarrow$ 0.70 $\rightarrow$ 0.78 as W grows). The conclusion that $W=60$ is the right default for this problem is therefore an LSTM-driven result; a deployment that ships the baseline should *not* reuse the same window length.

VII. DISCUSSION

Three observations stand out. First, on regression alone the three models are close enough that, for a deployment that only needs a numerical CPU forecast, the linear baseline is the rational choice—it costs orders of magnitude less to train and serve than the LSTM. Second, when the task is spike detection (the binary classification head), the LSTM clearly wins once the operating point is tuned: $F_1^*=0.78$ at $H=5$ min with

TABLE IV
CONTEXT-LENGTH ABLATION AT $H=5$ MIN. MAE ON TEST SET; F_1^* AT THE TEST-SET F_1 -OPTIMAL THRESHOLD.

Model	W=12	W=30	W=60
Baseline MAE	4.570	4.615	4.688
XGBoost MAE	4.618	4.451	4.504
LSTM MAE	4.832	4.502	4.482
Baseline F_1^*	0.492	0.329	0.271
XGBoost F_1^*	0.351	0.324	0.487
LSTM F_1^*	0.678	0.702	0.777

72% precision and 85% recall is a deployable spike detector, whereas the linear baseline tops out near $F_1^*=0.27$ at the same horizon. Third, the AUC-ROC numbers and the F_1 -at-0.5 numbers tell radically different stories for the imbalance-aware models; reporting only one of them would be misleading. The practical recommendation is to publish both AUC and the F_1^* obtained on a validation-set threshold sweep, and to report the chosen threshold so that the result is reproducible.

VIII. RESPONSIBLE AND ETHICAL CONSIDERATIONS

The Alibaba Cluster Trace v2018 is publicly released, anonymised at the machine level, and contains no tenant content or personal data—only operational telemetry. A residual risk exists in co-residency settings: workload fingerprints can in principle support tenant identification across deployments, so derived models should not be redistributed in a form that allows trivial inversion to per-tenant behaviour. The stratified sample over-represents bursty and high-utilisation profiles relative to their natural frequency; this is methodologically correct for the comparison but means the absolute F_1^* values in Table III are not deployment expectations without a target-cluster calibration step.

The classification head is intended to drive automated scaling decisions, so the asymmetric cost of false negatives (missed warnings, SLO violations) versus false positives (precautionary actions) is an operational and economic question, not a hyperparameter. The threshold τ^* should be chosen by the deployment team against its own service objectives and published alongside the model so the operating point remains auditable. Compute cost is also relevant: the LSTM grid search and training consumed several GPU-hours per context length, on the order of a single-GPU day for the full ablation. For pure regression the linear baseline is within 0.5% MAE of the LSTM at orders of magnitude lower energy cost, and is the rational deployment choice when the classification head is not needed.

IX. CONCLUSION

This paper presented a horizon-stratified comparison of three forecasting models on the Alibaba Cluster Trace v2018, with explicit attention to behavioural-profile stratification, residual-target learning for tree models, and decision-threshold calibration. The LSTM with a 60-minute context window is the best classifier across all four horizons after threshold tuning, while the linear baseline remains a strong regression choice

at long horizons. The clearest deployment lesson is that the default $\tau=0.5$ cutoff is unsafe for any of the models tested here; per-model threshold calibration on the validation split is mandatory. Future work includes (i) tuning the threshold on the validation split rather than on the test set, to convert the F_1^* values reported here from upper bounds into deployable operating points, and (ii) extending the classification target to predict the time-to-spike rather than a fixed- H indicator.

USE OF GENERATIVE AI TOOLS

A large language model (Anthropic’s Claude) was used during this work as a coding and writing assistant. Its scope was limited to: reviewing and refactoring pipeline code, diagnosing model-formulation issues (specifically the XGBoost piecewise-constant collapse that led to the delta-target reformulation in Section V), generating boilerplate for figures and result aggregation, and improving the clarity of English prose. The model was *not* used to generate, fabricate, or edit any experimental result, table, or figure: every numerical value reported in this paper was produced by the released code on the public Alibaba Cluster Trace v2018 and can be reproduced from the linked repository. Final decisions on experimental design, model selection, threshold-tuning methodology, and the paper’s claims rest with the authors.

REFERENCES

- [1] J. Guo, Z. Chang, S. Wang, H. Ding, Y. Feng, L. Mao, and Y. Bao, “Who limits the resource efficiency of my datacenter: An analysis of Alibaba datacenter traces,” in *Proc. IWQoS*, 2019.
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [6] J. Davis and M. Goadrich, “The relationship between precision-recall and ROC curves,” in *Proc. ICML*, 2006, pp. 233–240.